DECISION GOVERNANCE INFRASTRUCTURE

UNITED STATES PATENT AND TRADEMARK OFFICE

PROVISIONAL PATENT APPLICATION

TITLE OF INVENTION:

CRYPTO-AGILITY PROVIDER ARCHITECTURE FOR POST-QUANTUM GOVERNANCE SIGNING WITH HYBRID CLASSICAL AND POST-QUANTUM KEY ENCAPSULATION, FAIL-SAFE DEGRADATION, AND ALGORITHM-BOUND RECEIPT VERIFICATION

Docket Number:	OMNIX-PAT-2026-011
Applicant / Inventor:	Harold Alberto Nunes Rodelo
Nationality:	United Kingdom
Entity Status:	Micro Entity (37 C.F.R. § 1.29)
Filing Basis:	35 U.S.C. § 111(b) — Provisional Application
Date Prepared:	April 19, 2026
Date of Filing:	April 19, 2026

This document constitutes a Provisional Patent Application filed pursuant to 35 U.S.C. § 111(b). It establishes a priority date and grants twelve (12) months from the filing date to submit a corresponding non-provisional application. This application has not been examined and does not confer patent rights independently. CONFIDENTIAL — NOT FOR DISTRIBUTION.

PROVISIONAL PATENT APPLICATION

OMNIX-PAT-2026-011

Title: CRYPTO-AGILITY PROVIDER ARCHITECTURE FOR POST-QUANTUM GOVERNANCE SIGNING WITH HYBRID CLASSICAL AND POST-QUANTUM KEY ENCAPSULATION, FAIL-SAFE DEGRADATION, AND ALGORITHM-BOUND RECEIPT VERIFICATION

Inventor: Harold Alberto Nunes Rodelo

Applicant: OMNIX QUANTUM LTD

Filing Basis: 35 U.S.C. § 111(b)

Entity Status: Micro Entity

Date Prepared: April 19, 2026

Date of Filing: April 19, 2026

Related Applications: OMNIX-PAT-2026-001 (Governance Control Architecture), OMNIX-PAT-2026-003 (PQC Auth), OMNIX-PAT-2026-010 (Transparency Chain)

FIELD OF THE INVENTION

The present invention relates to cryptographic infrastructure for automated governance systems, and more particularly to a crypto-agility provider architecture that abstracts post-quantum and classical signing algorithms behind a uniform interface, enabling algorithm substitution through configuration without code modification; and to a hybrid key encapsulation mechanism combining classical elliptic-curve key exchange with post-quantum key encapsulation in a fail-safe degradation hierarchy, wherein the combined shared secret is derived using HKDF over the concatenated outputs of both mechanisms.

BACKGROUND

I. THE CRYPTOGRAPHIC TRANSITION PROBLEM IN GOVERNANCE SYSTEMS

Automated governance systems that issue cryptographically signed decision receipts face a unique version of the cryptographic transition problem: they must produce evidence records that remain verifiable for years or decades after issuance, across a period during which the underlying cryptographic assumptions may change fundamentally.

This problem has two distinct dimensions:

Dimension 1 — Algorithm Lifetime: A governance receipt signed today using a classical algorithm (e.g., ECDSA, Ed25519) may become forgeable within years as quantum computing capabilities advance. A regulator auditing receipts issued in 2026 from a 2035 perspective needs those receipts to remain cryptographically authentic. If the signing algorithm has been broken, the entire audit trail loses evidentiary value.

Dimension 2 — Algorithm Agility: The cryptographic landscape for post-quantum algorithms is evolving rapidly. NIST finalized its first post-quantum standards in 2024 (FIPS 203, 204, 205). Additional algorithms are under consideration. A governance system that hardcodes a specific post-quantum algorithm into its architecture faces the risk of that algorithm being deprecated, broken, or superseded — requiring architectural changes to every component that generates or verifies signed receipts.

Existing governance systems fail to address both dimensions:

1.1 Hardcoded Algorithms: Most governance systems hardcode their signing algorithm at the architectural level. Changing the algorithm requires modifying all signing code, all verification code, and all receipt parsing code throughout the system.

1.2 No Hybrid Security: Systems that have adopted post-quantum signing typically abandon classical signing entirely. This creates a different risk: if the post-quantum algorithm is broken or subject to an implementation vulnerability, there is no classical fallback. Conversely, systems that retain only classical signing have no protection against quantum computing attacks.

1.3 Algorithm Substitution Attacks: Systems that do not bind the signing algorithm identifier into the signed receipt payload are vulnerable to algorithm substitution attacks: an adversary replaces a valid post-quantum signature with a forged classical signature, claims the receipt was signed with the classical algorithm, and presents the forged receipt as authentic.

1.4 No Graceful Degradation: Systems that require both classical and post-quantum primitives fail completely when either component is unavailable (e.g., due to library unavailability in a constrained environment). A governance system that fails to produce signed receipts because a cryptographic library is unavailable fails its primary function.

The present invention addresses all four inadequacies through the Crypto-Agility Provider Architecture and Hybrid KEM.

SUMMARY OF THE INVENTION

The present invention provides two complementary systems:

System A — Crypto-Agility Provider Architecture: An abstraction layer that decouples signing algorithm selection from the governance pipeline components that generate and verify signed receipts. The architecture enables algorithm substitution through environment configuration without any modification to the governance pipeline code. The algorithm identifier is bound into the signed receipt payload, preventing algorithm substitution attacks. A provider registry enables simultaneous deployment of multiple algorithm providers, enabling backward-compatible verification of receipts signed under any registered algorithm.

System B — Hybrid Key Encapsulation Mechanism (Hybrid KEM): A key encapsulation mechanism that combines X25519 elliptic-curve Diffie-Hellman (classical) with Kyber-768 key encapsulation (post-quantum, NIST FIPS 203) in a hybrid construction, deriving a single combined shared secret using HKDF over the concatenated component secrets. The mechanism implements a four-tier fail-safe degradation hierarchy that gracefully degrades when either component is unavailable.

DETAILED DESCRIPTION OF THE PREFERRED EMBODIMENTS

I. CRYPTO-AGILITY PROVIDER ARCHITECTURE (ADR-043)

I.A. Design Principle

The central design principle of the Crypto-Agility Provider Architecture is that the signing algorithm used by the governance pipeline should be a deployment configuration parameter, not an architectural constant. A governance pipeline component that needs to sign a receipt calls `provider.sign(message, key)` and does not know or care which specific algorithm is being used. The algorithm is determined by the active provider, which is resolved from configuration at deployment time.

This design enables:

- **Algorithm substitution without code changes:** Deploying a different signing algorithm requires changing an environment variable (in one embodiment: `ACTIVE_SIGNING_PROVIDER`), not modifying any pipeline code.
- **Multi-algorithm verification:** A provider registry maintains all registered algorithms, enabling correct verification of receipts signed under any registered algorithm — including receipts signed under algorithms that are no longer the active signing algorithm.
- **Forward compatibility:** New algorithm providers can be registered without modifying existing providers or the pipeline components that use them.

I.B. Provider Interface

The CryptoProvider abstract interface defines the contract that all signing algorithm providers must implement. In one embodiment, the interface comprises:

- **provider_id():** Returns a unique string identifier for the algorithm (e.g., "dilithium3", "dilithium5", "ed25519"). This identifier is embedded in signed receipt payloads.
- **algorithm_name():** Returns a human-readable algorithm name for documentation and audit purposes.
- **generate_keypair():** Generates a public/private keypair for the algorithm. Returns a tuple of (public_key_bytes, private_key_bytes) or None if the algorithm is unavailable.
- **sign(message, secret_key):** Signs a message with the provided secret key. Returns raw signature bytes or None on failure.
- **verify(signature, message, public_key):** Verifies a signature against a message and public key. Returns a boolean.
- **serialize_public_key(public_key):** Serializes a public key to a string representation for storage and transmission.
- **deserialize_public_key(data):** Deserializes a public key from its string representation.

All providers implement this identical interface. Components that generate or verify signed receipts interact exclusively with this interface — they never interact with the underlying cryptographic library directly.

I.C. Provider Registry

The Provider Registry maintains a mapping from provider_id strings to CryptoProvider instances. The registry enables:

- **Active provider resolution:** The registry resolves the active signing provider from configuration (in one embodiment: the `ACTIVE_SIGNING_PROVIDER` environment variable or the `PQC_SIGNING_LEVEL` environment variable for backward compatibility).
- **Verification dispatch:** When verifying a receipt, the registry extracts the provider_id from the receipt metadata and dispatches verification to the corresponding registered provider, regardless of which provider is currently active for signing. This ensures that receipts signed under any historically registered algorithm remain verifiable.
- **Provider availability checking:** The registry can report the availability status of each registered provider, enabling the system to warn about unavailable providers before they are needed.

In one embodiment, the default provider registry includes three providers:

- **"dilithium3":** Dilithium-3 (ML-DSA-65, FIPS 204) — enterprise baseline
- **"dilithium5":** Dilithium-5 (ML-DSA-87, FIPS 204) — high-assurance deployments
- **"ed25519":** Ed25519 (classical) — development and test environments

In other embodiments, any number of providers implementing the CryptoProvider interface may be registered, including future post-quantum algorithms as they are standardized.

I.D. Algorithm-Bound Receipt Payload

A critical security property of the architecture is that the provider_id is bound into the signed receipt payload before signing. The signed content includes, among other fields:

```
{ ..., "signing_provider": "dilithium3", ... }
```

This binding prevents algorithm substitution attacks: an adversary cannot replace a valid post-quantum signature with a forged classical signature and claim the receipt was signed with a classical algorithm, because the provider_id field within the signed payload would not match the classical provider. Any mismatch between the provider_id in the signed payload and the algorithm used to produce the signature is detected at verification time.

I.E. Configuration-Driven Algorithm Selection

In one embodiment, the active signing provider is resolved as follows:

- If `ACTIVE_SIGNING_PROVIDER` environment variable is set, use the corresponding registered provider.

- Otherwise, if `PQC_SIGNING_LEVEL` is set ("high" → dilithium5, "standard" → dilithium3), resolve accordingly for backward compatibility.
- If neither is set, default to the enterprise baseline provider (dilithium3 in the preferred embodiment).

In other embodiments, the active provider may be resolved from any configuration source: database settings, encrypted configuration files, hardware security module policies, or operator-provided configuration at deployment time.

II. HYBRID KEY ENCAPSULATION MECHANISM (ADR-042)

II.A. Design Principle

The Hybrid KEM combines two independent key encapsulation mechanisms operating on orthogonal mathematical assumptions:

- **X25519 (classical ECDH):** Security based on the elliptic curve discrete logarithm problem. Broken by quantum computers running Shor's algorithm but computationally secure against classical adversaries.
- **Kyber-768 (post-quantum KEM, NIST FIPS 203):** Security based on the Module Learning With Errors (MLWE) problem. Designed to be secure against quantum computing attacks but subject to the risk of classical cryptanalytic breakthroughs.

By combining both mechanisms, the Hybrid KEM provides mutual insurance: if either mechanism is broken (classically or quantumly), the other mechanism still protects the shared secret. An adversary must simultaneously break both mechanisms to compromise the combined shared secret.

II.B. Combined Secret Derivation

The combined shared secret is derived using HKDF (HMAC-based Key Derivation Function) over the concatenated component secrets:

combined_secret = HKDF(kyber_secret || ecdh_secret, info="OMNIX-ADR042-HybridKEM-v1")

where kyber_secret is the shared secret produced by the Kyber-768 KEM, ecdh_secret is the shared secret produced by X25519 ECDH, || denotes concatenation, and the HKDF info label is bound to the specific protocol version to prevent cross-protocol attacks.

In one embodiment, HKDF is instantiated with SHA-256 and produces a 32-byte combined shared secret. In other embodiments, any secure key derivation function may be substituted, provided that it takes both component secrets as input and produces a single combined secret whose security depends on the security of both inputs.

Security property of HKDF combination: If kyber_secret is computationally indistinguishable from random (Kyber not broken), then combined_secret is computationally indistinguishable from random regardless of ecdh_secret. Conversely, if ecdh_secret is computationally indistinguishable from random (X25519 not broken by quantum), then combined_secret is computationally indistinguishable from random regardless of kyber_secret.

II.C. Four-Tier Fail-Safe Degradation Hierarchy

The Hybrid KEM implements a four-tier fail-safe degradation hierarchy that ensures the system can continue to produce shared secrets even when one or both cryptographic libraries are unavailable:

Tier	Condition	Mode	Security Level
Tier 1	Both Kyber and X25519 available	HYBRID	Quantum-resistant + classical
Tier 2	Only Kyber available	KYBER_ONLY	Quantum-resistant
Tier 3	Only X25519 available	ECDH_ONLY	Classical only
Tier 4	Neither available	NONE	Logged as critical failure

At Tier 4, the System logs a critical failure and does not produce a shared secret. The governance pipeline must treat this as a blocking condition — not as a pass-through.

The mode identifier is included in the key bundle metadata, enabling the receiving side to verify that the expected tier was used and to reject key bundles produced at lower-than-required security tiers.

II.D. Keypair Generation and Encapsulation

The System generates hybrid keypair bundles containing public and private components for each available mechanism:

Encapsulation (sender): The sender takes the recipient's public key bundle, encapsulates a random Kyber shared secret using the Kyber public key, computes the X25519 shared secret using the recipient's X25519 public key and an ephemeral X25519 keypair, combines both secrets using HKDF, and returns the combined shared secret alongside the ciphertext bundle (Kyber ciphertext + ephemeral X25519 public key).

Decapsulation (recipient): The recipient takes the ciphertext bundle and their private key bundle, decapsulates the Kyber ciphertext using their Kyber private key, computes the X25519 shared secret using their X25519 private key and the sender's ephemeral X25519 public key, and combines both secrets using HKDF to reproduce the combined shared secret.

III. INTEGRATION OF PROVIDER ARCHITECTURE AND HYBRID KEM

The Crypto-Agility Provider Architecture and the Hybrid KEM serve complementary functions within the OMNIX governance infrastructure:

Provider Architecture governs how governance receipts are signed and verified — it provides algorithm agility for the signature scheme used to seal governance decisions.

Hybrid KEM governs how session keys are established for secure communication between governance pipeline components and client systems — it provides hybrid security for the key exchange that protects data in transit.

Together, they implement a governance infrastructure that is secure against both classical and quantum adversaries at two distinct levels: the communication layer (Hybrid KEM) and the evidence layer (Provider Architecture + Transparency Chain).

IV. ALTERNATIVE EMBODIMENTS

4.1 Additional Provider Algorithms: The Provider Architecture supports registration of any signing algorithm implementing the CryptoProvider interface, including future NIST post-quantum standards (e.g., FALCON, SPHINCS+) and domain-specific signature schemes required by specific regulatory frameworks.

4.2 Hardware Security Module Integration: In one embodiment, individual providers delegate key storage and signing operations to a Hardware Security Module (HSM), with the HSM provider implementing the same CryptoProvider interface as software providers. This enables deployment in high-assurance environments requiring HSM-backed key management without architectural changes.

4.3 Multi-Signature Receipts: In one embodiment, governance receipts are signed by multiple providers simultaneously (e.g., both dilithium3 and ed25519), with all signatures included in the receipt. This enables verification by parties that support different subsets of the provider registry.

4.4 Alternative KEM Components: The Hybrid KEM framework supports substitution of either component: X25519 may be replaced by X448 or any other elliptic-curve Diffie-Hellman scheme; Kyber-768 may be replaced by any NIST-approved or future KEM algorithm. The HKDF combination mechanism is preserved regardless of component substitution.

4.5 Three-Component Hybrid: In one embodiment, a third KEM component is added (e.g., a lattice-based scheme different from Kyber), producing a three-way HKDF combination for additional algorithmic diversity.

4.6 Cross-Domain Application: The Provider Architecture and Hybrid KEM are equally applicable in any domain requiring quantum-resistant cryptographic infrastructure with algorithm agility: clinical data exchange, autonomous vehicle communication, industrial control system governance, energy grid management, and defense-grade classified system governance.

CLAIMS

Claim 1 (Broad — Provider Architecture) — A computer-implemented crypto-agility provider architecture for automated governance systems, comprising: a provider interface defining a uniform contract for signing algorithm implementations comprising keypair generation, message signing, and signature verification operations; a provider registry mapping algorithm identifiers to provider implementations; and an active provider resolver that selects the signing algorithm from deployment configuration without requiring modification of governance pipeline components; wherein governance pipeline components interact exclusively with the provider interface and are independent of the specific signing algorithm in use.

Claim 2 (Broad — Algorithm Binding) — The system of Claim 1, wherein the algorithm identifier of the active provider is embedded within the signed payload of every governance receipt before signing, and wherein verification of a receipt dispatches to the provider corresponding to the embedded algorithm identifier, preventing algorithm substitution attacks in which a valid signature under one algorithm is replaced with a forged signature claimed to be from a different algorithm.

Claim 3 (Specific — Configuration-Driven Selection) — The system of Claim 1, wherein the active signing provider is selected from the provider registry based on a deployment configuration value, and wherein changing the active signing algorithm requires only changing said configuration value with no modification to governance pipeline code.

Claim 4 (Broad — Backward Verification) — The system of Claim 1, wherein the provider registry retains all historically registered algorithm providers, enabling verification of governance receipts signed under any previously active algorithm regardless of which algorithm is currently active for signing.

Claim 5 (Broad — Hybrid KEM) — A computer-implemented hybrid key encapsulation mechanism comprising: a post-quantum key encapsulation component operating on a post-quantum mathematical hardness assumption; a classical key agreement component operating on a classical mathematical hardness assumption; and a key derivation function that derives a single combined shared secret from the concatenated outputs of both components; wherein the combined shared secret is computationally indistinguishable from random if either component's output is computationally indistinguishable from random.

Claim 6 (Specific — HKDF Combination) — The system of Claim 5, wherein, in one embodiment, the combined shared secret is derived as HKDF(kyber_secret || ecdh_secret, info=protocol_label), wherein kyber_secret is produced by a Kyber-768 key encapsulation mechanism as specified in NIST FIPS 203, ecdh_secret is produced by X25519 elliptic-curve Diffie-Hellman, and the protocol label binds the derivation to the specific protocol version.

Claim 7 (Broad — Fail-Safe Degradation) — The system of Claim 5, wherein the hybrid key encapsulation mechanism implements a fail-safe degradation hierarchy comprising at least: a first tier in which both classical and post-quantum components are available and the combined secret is derived from both; a second tier in which only the post-quantum component is available and the shared secret is derived from the post-quantum component alone; and a third tier in which only the classical component is available and the shared secret is derived from the classical component alone; wherein each tier's mode identifier is recorded in the key bundle metadata.

Claim 8 (Specific — Mode Binding) — The system of Claim 7, wherein the mode identifier indicating which degradation tier was used is included in the key bundle metadata, enabling the receiving party to verify that the expected security tier was used and to reject key bundles produced at security tiers below a configured minimum.

Claim 9 (Broad — Integration) — A computer-implemented cryptographic infrastructure for automated governance systems comprising: a crypto-agility provider architecture that enables configuration-driven algorithm substitution for governance receipt signing without code modification; a hybrid key encapsulation mechanism that combines classical and post-quantum key agreement with fail-safe degradation; and an algorithm binding mechanism that embeds the provider identifier in signed receipt payloads to prevent algorithm substitution attacks; wherein the infrastructure maintains cryptographic evidence integrity across the quantum computing transition period.

Claim 10 (Broad — Method) — A computer-implemented method for algorithm-agile post-quantum governance receipt signing, comprising: resolving an active signing provider from deployment configuration; embedding the active provider's algorithm identifier in a governance receipt payload; signing the payload using the active provider's signing operation;

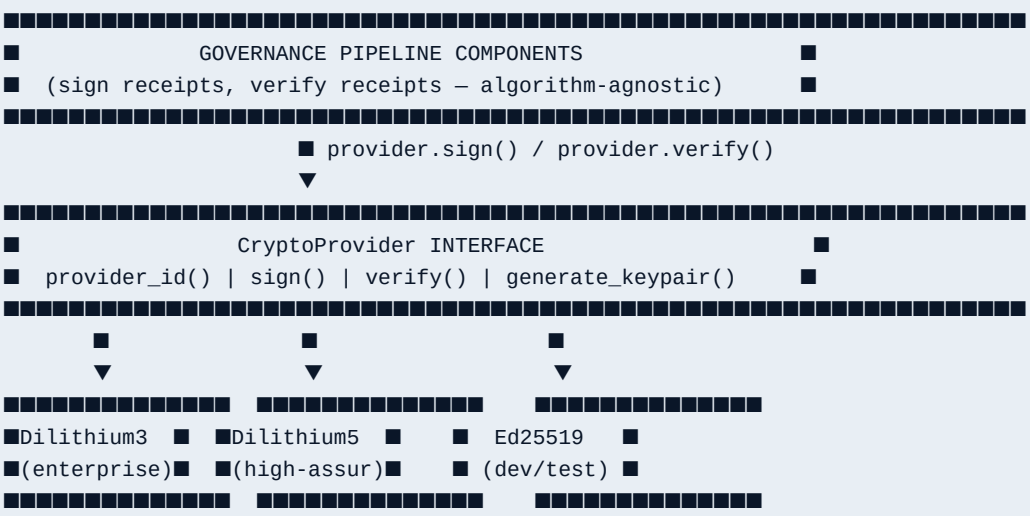
persisting the signed receipt with the embedded algorithm identifier; and verifying the receipt by extracting the algorithm identifier and dispatching verification to the corresponding registered provider.

ABSTRACT

A crypto-agility provider architecture abstracts post-quantum and classical signing algorithms behind a uniform provider interface, enabling algorithm substitution in automated governance systems through environment configuration without modification of governance pipeline code. A provider registry maintains all registered algorithm providers — including post-quantum schemes (Dilithium-3 ML-DSA-65, Dilithium-5 ML-DSA-87) and classical fallbacks (Ed25519) — enabling backward-compatible verification of receipts signed under any historically registered algorithm. The signing algorithm identifier is embedded within the signed payload of every governance receipt, preventing algorithm substitution attacks. A complementary hybrid key encapsulation mechanism combines X25519 classical ECDH with Kyber-768 post-quantum KEM (NIST FIPS 203), deriving a single combined shared secret using HKDF over concatenated component secrets (in one embodiment: HKDF(kyber_secret || ecdh_secret)). A four-tier fail-safe degradation hierarchy gracefully degrades when either cryptographic library is unavailable. The combined architecture provides quantum-resistant cryptographic infrastructure at both the communication layer (hybrid KEM) and the evidence layer (provider architecture) for governance systems in financial trading, clinical decision support, autonomous robotics, energy management, and any regulated domain requiring cryptographic evidence integrity across the quantum computing transition. To the inventor's knowledge, this is the first system combining a configuration-driven crypto-agility provider registry with algorithm-bound governance receipt signing and a hybrid classical+post-quantum KEM with fail-safe degradation hierarchy specifically for automated decision governance infrastructure.

DRAWINGS

FIG. 1 — Crypto-Agility Provider Architecture



Active provider selected via: ACTIVE_SIGNING_PROVIDER env var
Receipt payload includes: { "signing_provider": "dilithium3", ... }
Verification dispatches by: receipt.signing_provider → registry lookup

FIG. 2 — Algorithm-Bound Receipt (Anti-Substitution)

```
SIGNED PAYLOAD:
{
  "receipt_id": "...",
```

```
    "decision": "PASS",
    "signing_provider": "dilithium3",   ← bound into signed content
    "checkpoint_results": {...},
    "ts_utc": "2026-04-19T..."
  }
  signature = dilithium3.sign(payload, secret_key)

  ATTACK ATTEMPT: replace signature with ed25519 forgery
  DETECTED: payload.signing_provider = "dilithium3" ≠ "ed25519"
            → verification fails
```

FIG. 3 — Hybrid KEM Fail-Safe Degradation

Kyber Available?	X25519 Available?	Mode	Security
YES	YES	HYBRID	PQC + Classical
YES	NO	KYBER_ONLY	PQC only
NO	YES	ECDH_ONLY	Classical only
NO	NO	NONE	CRITICAL FAILURE

HYBRID combined_secret = HKDF(kyber_secret || ecdh_secret)
Mode embedded in key bundle → receiver enforces minimum tier